APPENDIX 1

CampusTrack – A Smart In-Out Digital Registry System

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

Aditya Aryan — Roll No. A21
Om Aswar — Roll No. A23
Sneha Ainapure — Roll No. A22
Sadichha Jadhav — Roll No. A58

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

IN

Artificial Intelligence and Machine Learning



KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING KOLHAPUR
(EMPOWERED AUTONOMOUS)

A.Y. 2026-27

APPENDIX 2

KOLHAPUR INSTITUTE OF TECHNOLOGY'S COLLEGE OF ENGINEERING KOLHAPUR (EMPOWERED AUTONOMOUS)

CERTIFICATE

This is to certify that the Seminar/ Project report entitled, **CampusTrack – A Smart In-Out Digital Registry System** submitted by **Aditya Aryan (Roll No. A21), Om Aswar (Roll No. A23), Sneha Ainapure (Roll No. A22), Sadichha Jadhav (Roll No. A58)**, in partial fulfillment for the award of the degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING** in **Artificial Intelligence and Machine Learning** at Kolhapur Institute of Technology's College of Engineering Kolhapur (Empowered Autonomous), Maharashtra, INDIA, is a record of his / her own work carried out under my / our supervision and guidance.

SIGNATURE
SUPERVISOR

SIGNATURE
HEAD OF THE DEPARTMENT

APPENDIX 3

KOLHAPUR INSTITUTE OF TECHNOLOGY'S COLLEGE OF ENGINEERING KOLHAPUR (EMPOWERED AUTONOMOUS)

DECLARATION

I hereby declare that the Seminar/ Project entitled, **CampusTrack – A Smart In-Out Digital Registry System** submitted to Kolhapur Institute of Technology's College of Engineering Kolhapur (Empowered Autonomous), Maharashtra, INDIA in the partial fulfillment of the award of the Degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING in Artificial Intelligence and Machine Learning** is a Bonafide work carried out by me. The material contained in this Seminar/ Project has not been submitted to any University or Institution for the award of any degree.

Aditya Aryan	**2526UAI147**	A21
Om Aswar	**2526UAI135**	A23
Sneha Ainapure	**2526UAI097**	A22
Sadichha Jadhav	**2526UAI024**	A58

Place:
Date:

APPENDIX 4

KOLHAPUR INSTITUTE OF TECHNOLOGY'S COLLEGE OF ENGINEERING KOLHAPUR (EMPOWERED AUTONOMOUS)

ACKNOWLEDGEMENT

We would like to express our deep gratitude towards **Mrs. Amruta Pradip Chougule** and **Dr. Uma Gurav**, Head of Department, for their constant encouragement and guidance. Their support and cooperation throughout the project work have been of immense help to us. We express our sincere thanks to all the teaching and non-teaching staff and all those who have directly or indirectly helped in making the project a success.

Aditya Aryan	**2526UAI147**	A21
Om Aswar	**2526UAI135**	A23
Sneha Ainapure	**2526UAI097**	A22
Sadichha Jadhav	**2526UAI024**	A58

Place:
Date:

APPENDIX 5

TABLE OF CONTENTS

CERTIFICATE 1
TABLE OF CONTENTS 2
1. Abstract 6
2. Introduction 6
3. Problem Statement 8
4. Objectives 8
5. Literature Survey / Existing System 9
6. Proposed System 9
7. Scope of the Project 10
In Scope 10
Out of Scope 10
8. Methodology / System Design 11
Technology Stack 11
System Architecture 11
Modules 13
9. Hardware and Software Requirements 14
10. Expected Outcome 14
11. Applications 16
12. Timeline / Work Plan 17
13. References 17

1. Abstract

Attendance and in-out registers in hostels, classrooms and apartment complexes are still largely maintained on paper. A paper register cannot verify who actually signed it, cannot be inspected in real time, and cannot stop one person from signing in for another (proxy attendance). CampusTrack – A Smart In-Out Digital Registry System addresses this gap with a Flask-based web application in which every check-in or check-out must pass three independent checks: a signed QR code scan at the checkpoint, a live face match against the user's registered photo using OpenCV (Haar cascade detection and LBPH recognition), and a GPS geofence check based on the haversine distance. Every scan attempt is stored as an attendance log entry, and attempts that fail any check are flagged for administrator review instead of being silently dropped.

The platform supports many independent institutes on a single deployment through three role-based logins (superuser, admin and user), a drag-and-drop custom registration form builder, batch QR room generation, and a live in/out dashboard. It is built with Flask, Flask-SQLAlchemy over SQLite, Flask-Login, OpenCV, qrcode and itsdangerous, requires no paid APIs or extra hardware, and is already deployed end-to-end at campustrack.adityaaryan.me. The project shows that low-cost, multi-factor attendance verification can replace manual registers and give administrators a real-time headcount.

2. Introduction

Web development combined with computer vision has made it practical to move everyday administrative work from paper to software. Attendance and in-out tracking is one of the most common examples: schools, colleges, hostels, offices and residential societies all need to know who is present and who is inside premises at a given moment. Most of them still rely on a notebook or a printed register that is signed by hand.

Modern smartphones already carry the three things needed to do this better: a camera, a GPS receiver and a web browser. A QR code placed at a checkpoint can prove that the person is at that checkpoint, the camera can confirm who the person is, and the GPS reading can confirm where they are. Individually each of these checks can be bypassed (a QR photo can be forwarded, a location can be shared), but together they make impersonation considerably harder.

The motivation for this project came from the hostel notebook: a system that is easy to fill in dishonestly, impossible to query live, and hard to audit afterwards. CampusTrack replaces it with a single verifiable digital flow that works in an ordinary mobile browser, without special hardware or paid third-party services.

3. Problem Statement

Paper registers used for attendance and in-out tracking cannot verify the identity of the person signing, cannot be checked in real time, and cannot prevent one person from signing in on behalf of another. As a result the records are manual, error-prone, scattered and unreliable, and administrators have no live headcount of who is actually inside the premises. There is a need for a low-cost, hardware-free system that verifies both identity and location before a presence is logged.

4. Objectives

- To replace paper registers with a verifiable digital check-in / check-out flow.
- To guarantee identity and location before presence is logged, by combining a signed QR scan, a live face match (OpenCV LBPH) and a GPS geofence check.
- To give administrators a live in/out dashboard with per-room occupancy instead of a daily count.
- To support many independent institutes (colleges, hostels, apartments, companies) on a single deployment with role-based access and strict data isolation.
- To keep the stack low-cost – no paid APIs and no extra hardware – and to deploy the system end-to-end so it can be used in practice.

5. Literature Survey / Existing System

The following existing papers and systems were reviewed. Most solve one part of the problem (face recognition, or QR with location) but not identity, presence and location together in a multi-institute platform.

Research Paper (Published Year)	Approach / Method	Dataset / Data Used	Key Findings Limitations /	Research Gap
Rathi et al., Multi-Facial Automated Attendance System using Haar Cascade, LBPH and OpenCV (2023) [1]	Haar cascade face detection and LBPH face recognition in OpenCV to mark classroom attendance automatically.	Enrolled student face images captured with a camera.	Automates attendance and counts the people present in a class. Face is the only factor; no proof of place or session.	No second factor (QR / location); no multi-institute management or live in/out dashboard.
Fraud Mitigation in Attendance Monitoring Systems using Dynamic QR Code, Geofencing and IMEI (IJACSA, 2023) [2]	Dynamic QR code that refreshes periodically and encodes venue coordinates, combined with geofencing and a device IMEI check.	Event venue coordinates and registered device identifiers.	Reduces proxy attendance through QR sharing and out-of-area scans. Identity is tied to a device, not to the person's face.	Anyone holding a registered device can mark attendance; no biometric identity check.
Revanth D. et al., Attendify: Cloud-Based Multi-Factor Attendance System using QR Code, Facial Recognition and Geofencing (IJERT, 2026) [3]	Session QR code / OTP generated by the teacher, plus facial recognition and GPS geofencing, hosted on the cloud.	Enrolled user photos, per-session QR/OTP, GPS coordinates.	Combining the three factors is reported to eliminate proxy attendance. Designed around teacher-initiated class sessions.	Class-session oriented; does not cover fixed checkpoints, in/out occupancy tracking, or many institutes with custom registration forms.
M. S. Sai, Real-Time Face Recognition Attendance System using OpenCV (IJERT, 2026) [4]	OpenCV with LBPH algorithm for real-time face detection and recognition to reduce proxy attendance.	Enrolled face images of students.	Real-time recognition with automatic attendance marking. Single-factor system.	No location or QR verification and no liveness check; limited administration features.
Traditional paper registers and card / RFID / fingerprint terminals (existing systems)	Handwritten signature, or dedicated hardware readers at the entrance.	Paper sheets or hardware-local logs.	Simple and familiar, but paper cannot verify identity or be queried live; hardware adds cost and maintenance.	Proxy sign-in is easy on paper; hardware is costly and per-site. CampusTrack needs only a phone and a browser.

6. Proposed System

CampusTrack is a Flask web application in which every check-in or check-out must pass three independent checks before it counts as verified. This closes the proxy-attendance gap that paper registers and single-factor systems leave open, without any extra hardware.

- **Signed QR scan per checkpoint:** each room gets its own QR code whose payload is signed with itsdangerous, so a forged or edited code is rejected without a database lookup.
- **Live face match:** a selfie captured at scan time is compared with the user's registered photo using OpenCV (Haar cascade detection and the LBPH recognizer).

- **GPS geofence check:** the phone's coordinates are compared with the room's location using the haversine formula and must fall inside the configured radius.
- **Nothing silently dropped:** every scan attempt is written to the attendance log; failed attempts are flagged for administrator review, and a 5-second cooldown prevents duplicate camera-fire entries.
- **Multi-institute, role-based access:** Admin, User and Superuser logins share one @role_required decorator; every admin query is scoped to the caller's own institute.
- **Admin tools:** drag-and-drop custom registration fields, batch room / QR generation, join-request approvals, and a live in/out dashboard.

Live deployment: https://campustrack.adityaaryan.me/ **Source code:**

https://github.com/Aditya-Aryan-6914/InOut-register

7. Scope of the Project

In Scope

- Admin signup that creates an institute and its first admin together; multi-institute support on one deployment.
- Three role-based logins (admin / user / superuser) with salted password hashes and safe ?next= redirects.
- User registration with institute join-password, custom fields, profile photo upload and admin approval of join requests.
- Drag-and-drop custom registration field builder.
- Room management with batch-generated, signed QR codes and per-room GPS location and geofence radius.
- Scan verification (QR + face + location), automatic check-in / check-out toggling and flagging of failed attempts.
- Live admin dashboard (registered users, currently in / out, check-ins today, per-room occupancy) and personal attendance history.
- Superuser portal to search, suspend / reactivate, change the plan of, or delete institutes.
- Deployment with Gunicorn and a live public site.

Out of Scope

- Liveness detection – the LBPH face match can currently be fooled by a printed photo.
- CSRF protection and access control on uploaded photos (tracked as top priority before large-scale real-world deployment).
- Time-rotating (dynamic) QR codes and GPS-spoofing detection.
- Native Android / iOS applications (the system is a mobile-friendly web application).
- Database migrations (the schema is currently created with db.create_all()) and online payment for subscription plans.
- Email / SMS notifications and deep-learning based face recognition models.

8. Methodology / System Design

Technology Stack

- **Programming Language(s):** Python 3.12, JavaScript, HTML / CSS (Jinja2 templates), SQL.
- **Framework(s) / Libraries:** Flask 3.1 (application factory + Blueprints), Flask-Login, Flask-SQLAlchemy / SQLAlchemy 2.0, OpenCV (opencv-contrib-python-headless), NumPy, Pillow, qrcode, itsdangerous, Werkzeug; front-end: SortableJS and html5-qrcode (vendored).
- **Database:** SQLite through Flask-SQLAlchemy (instance/inout.db).
- **Tools:** Git and GitHub, pytest, Gunicorn (WSGI server) and a web browser with camera and geolocation access.

System Architecture

CampusTrack follows a client–server architecture. The browser renders server-side Jinja pages, scans the QR code with html5-qrcode, captures a selfie from the camera and reads GPS coordinates through the Geolocation API. These three values are sent to the Flask application, which routes them through the Verification Engine and records the outcome in the database.

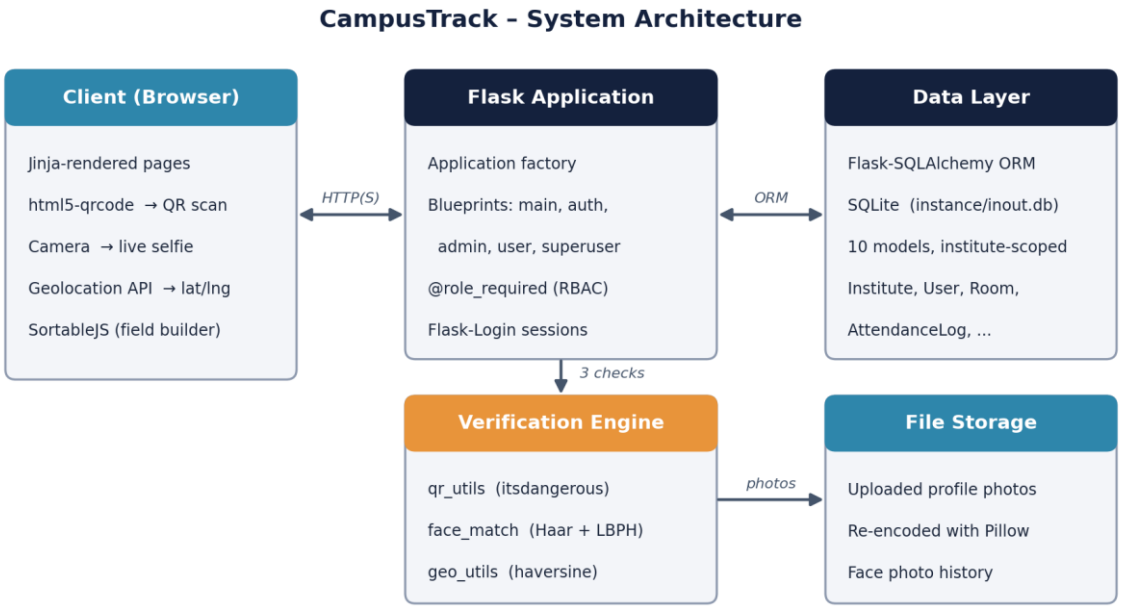


Figure 1: CampusTrack system architecture

Data flow of a scan: (1) the user scans a room QR, captures a selfie and reads GPS in the browser; (2) the browser POSTs {qr, photo, lat, lng} to /user/scan/verify; (3) the server verifies the signed QR, matches the face with OpenCV LBPH (distance ≤ 70 counts as a match) and checks the haversine distance against the room's geofence radius (default 100 m, configurable from 10 m to 1000 m); (4) one AttendanceLog row is inserted with the result of every factor;

(5) the server returns JSON {verified, event_type}; (6) the browser shows check-in / check-out, or “flagged for review”.

UML and database diagrams. The use case diagram shows the capabilities of the three roles, the class diagram shows the core domain model, the sequence diagram shows the scan verification flow, and the ER diagram shows the database relationships.

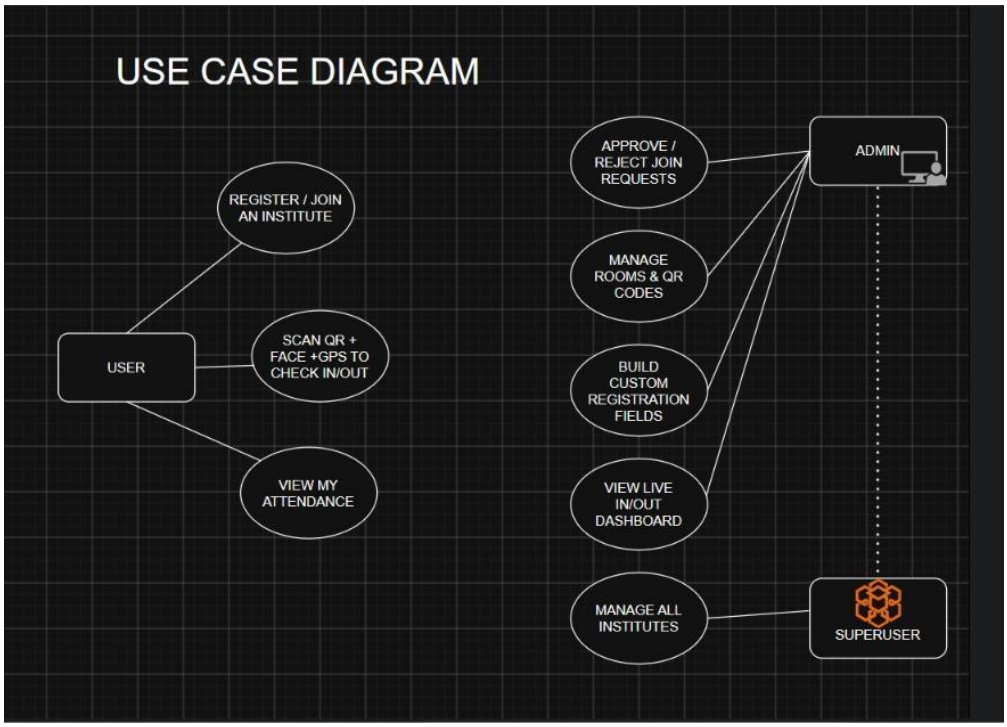


Figure 2: Use case diagram

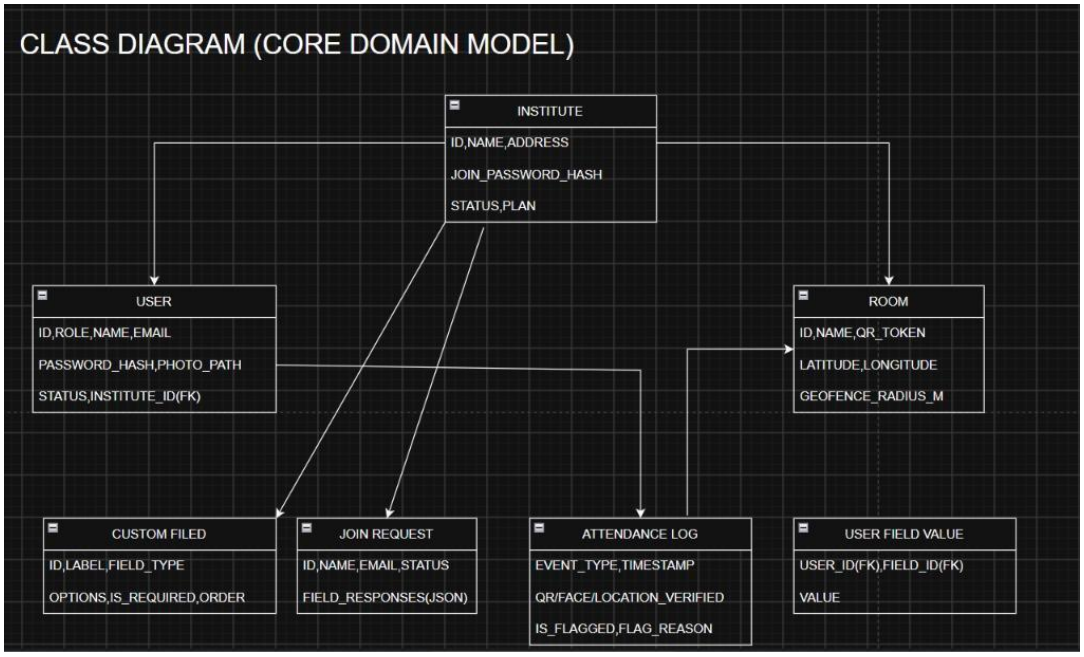


Figure 3: Class diagram (core domain model)

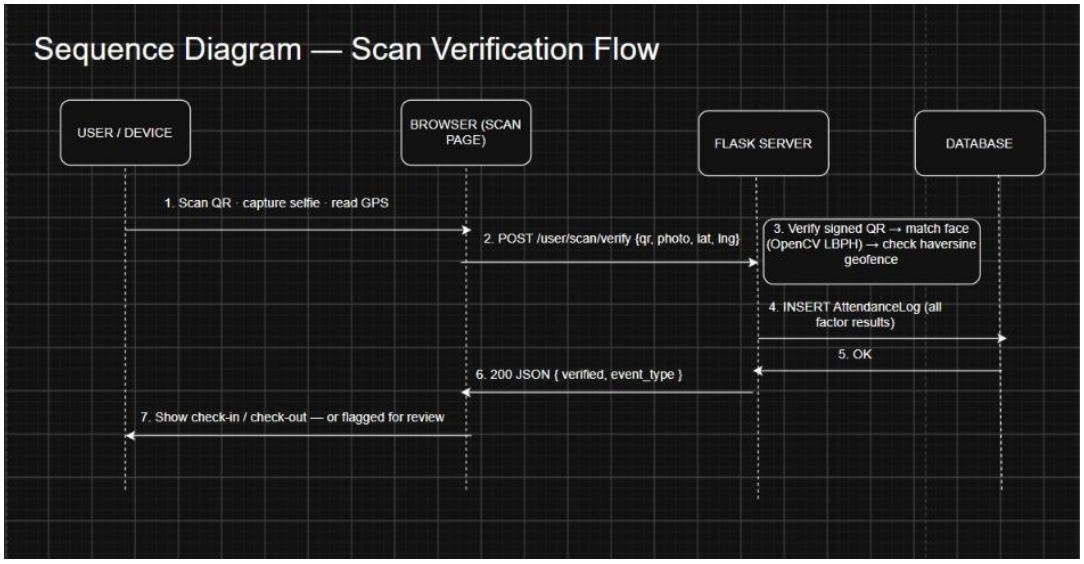


Figure 4: Sequence diagram – scan verification flow

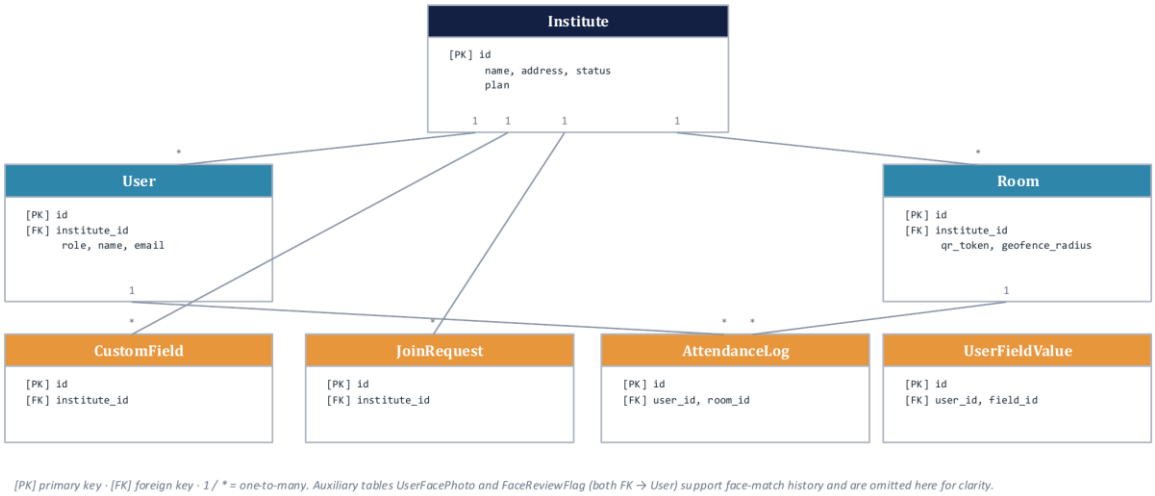


Figure 5: Entity-relationship diagram (UserFacePhoto and FaceReviewFlag omitted for clarity)

Modules

- **Auth & Access Control –** three separate login routes (admin / user / superuser) sharing one @role_required decorator, salted password hashes and safe ?next= redirects.
- **Admin Console –** institute dashboard, drag-and-drop custom-field builder, room / QR batch generator and join-request approvals.
- **User Portal –** 3-step registration with the institute join-password, profile photo upload, the live scan screen and personal attendance history.
- **Superuser Platform –** cross-institute view to search institutes, suspend / reactivate, change plan or delete an institute with a typed confirmation.
- **Verification Engine –** qr_utils (signed payloads), face_match (OpenCV Haar + LBPH) and geo_utils (haversine geofencing): the three checks behind every scan.
- **Data Layer –** 10 SQLAlchemy models (Institute, User, Room, CustomField, JoinRequest, UserFieldValue, AttendanceLog, UserFacePhoto, FaceReviewFlag, …); every admin query is scoped to the caller's institute_id.

9. Hardware and Software Requirements

Hardware Requirements	Software Requirements
Server: any PC / VPS with a dual-core processor (Intel i3 or above)	Operating System: Windows / Linux / macOS (server); Android / iOS (client)
RAM: 2 GB minimum (4 GB recommended for face matching)	Languages & runtime: Python 3.12 or newer, JavaScript
Storage: 1 GB free (application, SQLite database and uploaded photos)	Libraries: Flask, SQLAlchemy, Flask-Login, OpenCV, NumPy, Pillow, qrcode, itsdangerous (see requirements.txt); Gunicorn for production
Client device: smartphone or laptop with a camera, GPS / location service and internet access	Browser & database: Chrome / Edge / Firefox / Safari with camera and location permission; SQLite

10. Expected Outcome

The result of the project is a fully working, deployed web application. It demonstrates that every check-in and check-out can be verified with a signed QR scan, a live face match and a GPS geofence check, and that administrators get a live in/out dashboard instead of a daily count. The system supports multiple institutes with isolated data, role-based access, custom registration forms and batch-generated QR checkpoints, using only open-source libraries and no paid APIs.

Live project: https://campustrack.adityaaryan.me/ **GitHub repository:** https://github.com/Aditya-Aryan-6914/InOut-register

Key parameters implemented: LBPH match threshold of 70; default geofence radius of 100 m (10–1000 m configurable per room); 5-second scan cooldown; up to 500 rooms per institute; uploaded photos limited to 5 MB and re-encoded with Pillow.

Screenshots of the implemented system (taken from the deployed application):

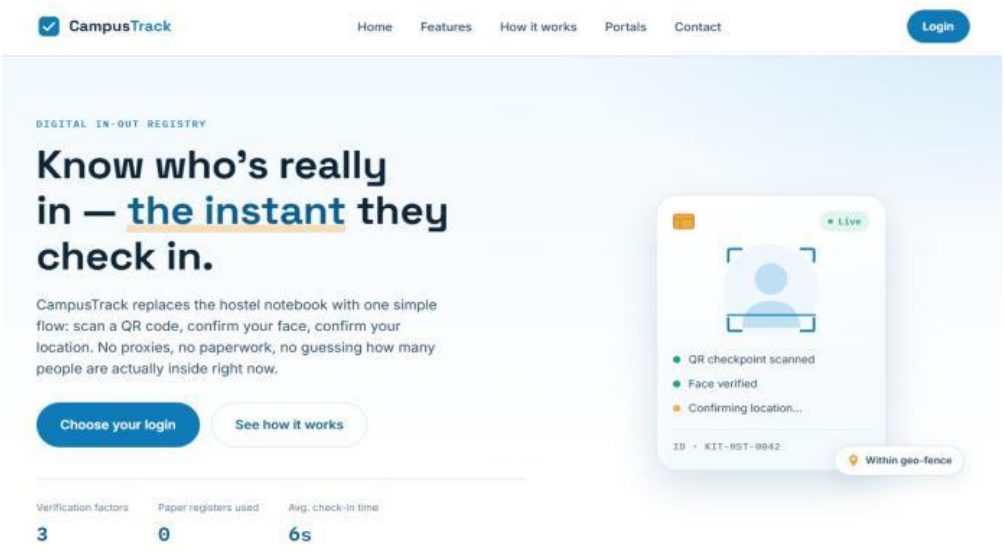


Figure 6: Landing page with live scan-flow preview

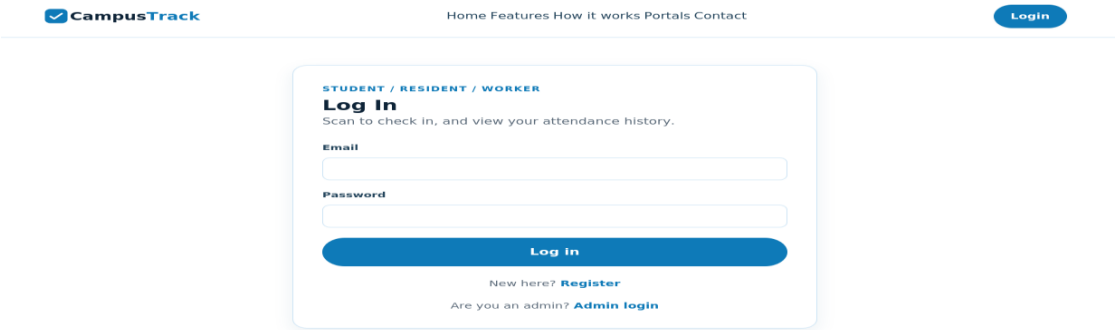


Figure 7: User login (students / residents / workers)

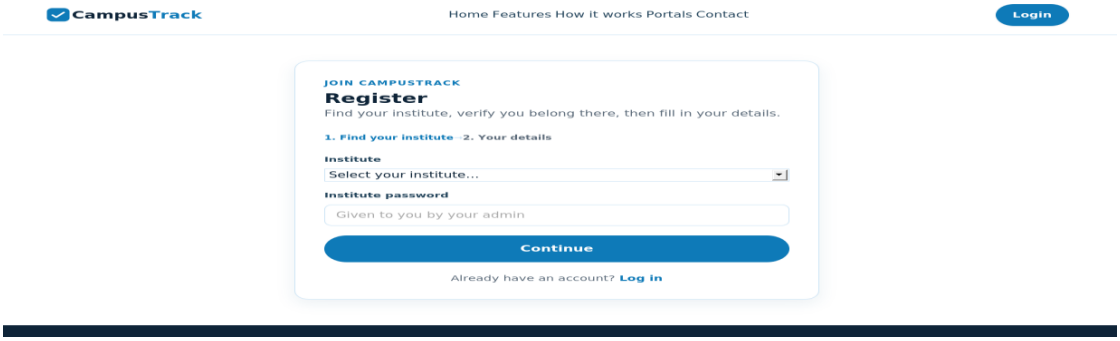


Figure 8: User registration – pick institute, join with shared password

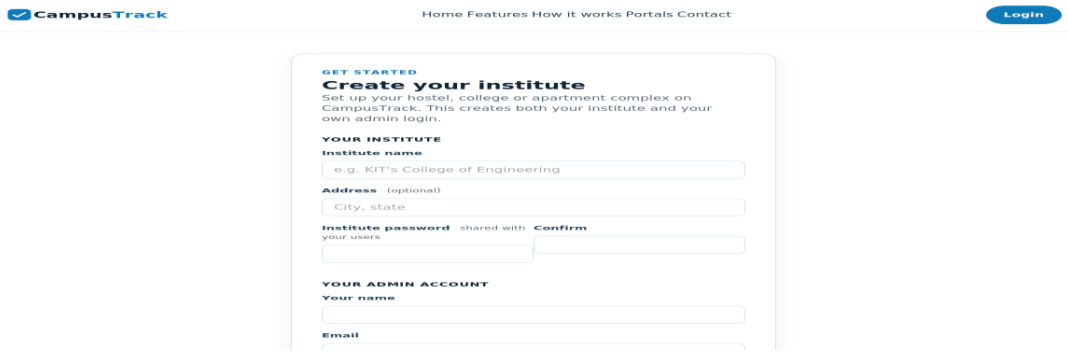


Figure 9: Admin signup – creates the institute and first admin together

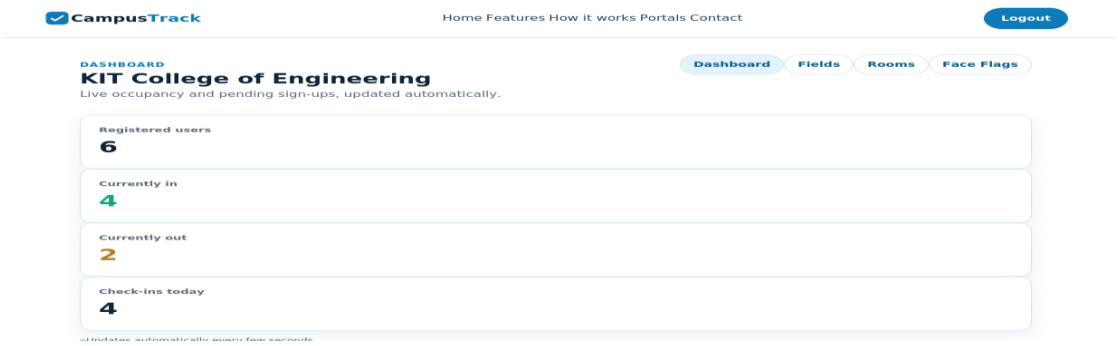


Figure 10: Admin dashboard – live in/out counts

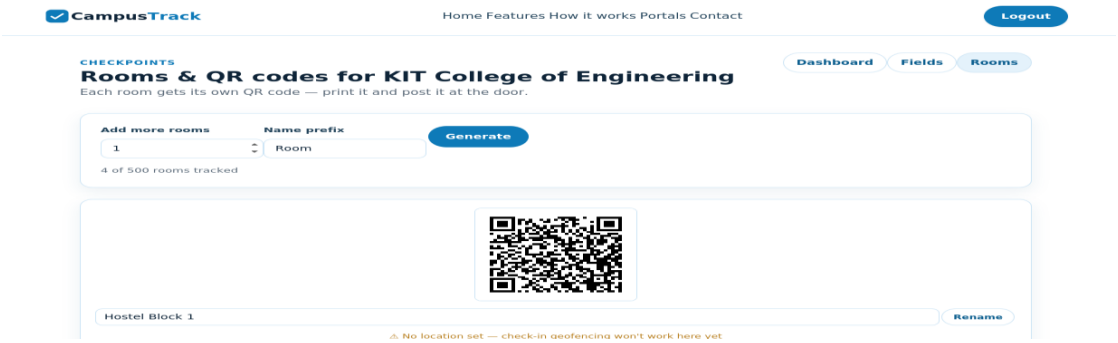


Figure 11: Rooms & QR codes – batch-generated, signed per checkpoint

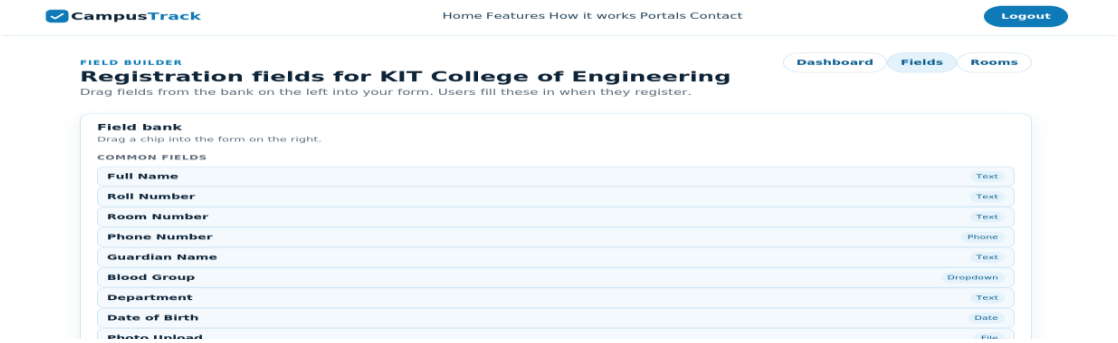


Figure 12: Custom field builder – drag-and-drop registration form

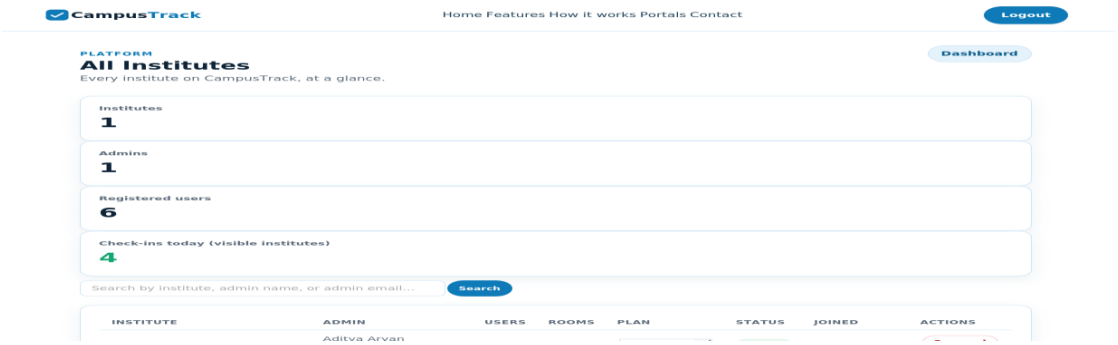


Figure 13: Superuser portal – every institute on the platform

11. Applications

- Hostels and PGs: digital in-out register for residents, with a live count of who is inside.
- Classrooms, labs and colleges: proxy-resistant attendance with a verified identity and location.

- Apartment complexes and housing societies: tracking entry and exit of residents and staff at gates and common areas.
- Offices, workshops and factories: verified shift check-in / check-out for workers across sites.
- Training institutes and multi-branch organisations: one deployment serving many independent institutes.

12. Timeline / Work Plan

Phase	Activity	Duration
Phase 1	Requirement analysis & literature survey	Week 1–2
Phase 2	System design & architecture	Week 3–4
Phase 3	Implementation / coding	Week 5–8
Phase 4	Testing & debugging	Week 9–10
Phase 5	Documentation & final submission	Week 11–12

13. References

[1] S. Rathi, S. Kapadnis, G. Sonar, G. Sonar and S. R. Vispute, "Multi-Facial Automated Attendance System using Haar Cascade, LBPH, and OpenCV-Based Face Detection and Recognition," IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10392248

[2] "Fraud Mitigation in Attendance Monitoring Systems using Dynamic QR Code, Geofencing and IMEI Technologies," Int. J. Advanced Computer Science and Applications (IJACSA), vol. 14, no. 4, 2023. [Online]. Available: https://thesai.org/Downloads/Volume14No4/Paper_104-Fraud_Mitigation_in_Attendance_Monitoring_Systems.pdf

[3] D. Revanth, K. Sathwik, N. Akshitha and K. Kolli, "Attendify: A Cloud-Based Multi-Factor Attendance System using QR Code, Facial Recognition and Geofencing," Int. J. Engineering Research & Technology (IJERT), vol. 15, no. 4, Apr. 2026. [Online]. Available: https://www.ijert.org/attendify-a-cloud-based-multi-factor-attendance-system-using-qr-code-facial-recognition-and-geofencing-ijertv15is040611

[4] M. S. Sai, "Real-Time Face Recognition Attendance System using OpenCV," Int. J. Engineering Research & Technology (IJERT), vol. 15, no. 4, Apr. 2026. [Online]. Available: https://www.ijert.org/real-time-face-recognition-attendance-system-using-opencv-ijertv15is042833

[5] Flask Documentation, Pallets Projects, https://flask.palletsprojects.com, Accessed on: 01 Oct 2026.

[6] OpenCV Documentation – Face Recognition with LBPH, https://docs.opencv.org, Accessed on: 01 Oct 2026.

[7] CampusTrack (live application), https://campustrack.adityaaryan.me/, Accessed on: 01 Oct 2026.

[8] CampusTrack / InOut-register source code, GitHub, https://github.com/Aditya-Aryan-6914/InOut-register, Accessed on: 01 Oct 2026.